A Practical Study of Git Version Control:

Tanmayee Vuppala

Abstract—This guide follows the visual, commit-tree-based learning style of Learn Git Branching and explains Git from first principles through branches, HEAD, relative references, merging, rebasing, remotes, origin/main, fetch, pull, push, remote tracking, refspecs, cherry-pick, interactive rebase, amend, tags, describe, and advanced parent selection. Each command is introduced with its purpose, syntax, a small mental model, and a visual commit graph where useful. Commands used only by the Learn Git Branching simulator are explicitly marked so they are not confused with ordinary Git commands.

Keywords—Git, GitHub, main, branch, HEAD, origin, remote, fetch, pull, push, merge, rebase, cherry-pick, reset, revert, tag,

I. THE GIT MENTAL MODEL

Git is a **distributed version control system (VCS)** used to track changes in files and manage different versions of a project. It allows developers to save their work, maintain a history of changes, experiment with different features using branches, and collaborate with others efficiently. Git is commonly used for software development and works with platforms such as GitHub to store and share repositories.Git mainly consists of four important domains: **Local Repository, Staging Area, Branching and Merging, and Remote Repository**. The **Local Repository** stores the project and its version history on your computer, while the **Staging Area** allows you to select and prepare changes before committing them. **Branching and Merging** help developers work on different features independently and later combine their changes. The **Remote Repository** stores the project online using platforms such as GitHub, allowing developers to push, pull, share, and collaborate on code.

A. Commits

A commit is a recorded snapshot of the tracked project state. A commit normally has a parent, which records where the work came from. The result is a history that can be represented as a directed graph.

```
git commit
```

Mental model: a new commit is created from the commit you are currently working on. The branch you are on normally moves forward to the new commit.

B. Branches

A branch is essentially a movable pointer to a commit. Creating a branch does not copy the entire project. It creates a new name pointing at the current commit.

```
git branch newImage
```

C. HEAD

HEAD is the symbolic name for the commit currently checked out. Normally HEAD points to a branch name, and that branch points to the current commit.

When you commit while on main, main moves to the new commit and HEAD continues to refer to main.

II. COMMITS AND BASIC NAVIGATION

A. Creating Commits

```
git commit
```

Use this to record the current staged changes as a new commit. Learn Git Branching focuses on the commit graph: each new commit is attached to the current commit.

B. Creating and Switching Branches

```
git branch <name>
```

Creates a branch pointer at the current commit without switching to it.

```
git checkout <name>
```

Switches HEAD to the specified branch.

```
git checkout -b <name>
```

Creates a new branch and switches to it in one operation.

```
git switch <name>
```

Modern Git also provides a switch for branch switching. Learn Git Branching notes that checkout was used throughout its lessons because switch was introduced later.

```
git switch -c <name>
```

Modern equivalent of creating and switching to a new branch.

C. Viewing History

```
git log
```

Shows commit history and is especially useful for finding commit hashes. In real repositories, commit hashes are much longer than the short IDs shown in the visualizer, but Git accepts a unique abbreviated hash.

III. THE MAIN BRANCH

main is a branch name, not a special kind of commit. In a repository, main is simply a reference that usually points to the commit representing the current main line of development.

If you create a feature branch from main, both names initially point to the same commit. New commits move only the branch you are currently on.

Important distinction: main is local when you are looking at your local repository. origin/main is a remote-tracking reference stored locally that represents the last known state of the remote main branch.

IV. MERGING BRANCHES

A. Basic Merge

```
git merge bugFix
```

Run this while on the branch that should receive the work. Git combines the histories. If the histories have diverged, the result can be a merge commit with two parents.

B. Fast-Forward Merge

If the branch being merged is already an ancestor of the current branch, Git can simply move the branch pointer forward instead of creating a new merge commit.

```
git checkout bugFix
git merge main
```

The Learn Git Branching lesson demonstrates this pointer movement when main is already ahead of bugFix.

C. Merge Conflicts

When two branches contain incompatible changes to the same part of a file, Git may stop and ask you to resolve the conflict. After resolving the files, the merge can be completed with a commit.

V. REBASE

A. Basic Rebase

```
git rebase main
```

Rebase takes the commits from the current branch that are not already in the target history, copies them, and places the copies on top of the target. The original commits remain as historical objects, while the rebased copies have new identities.

Git Branching describes rebasing as producing a cleaner, more linear-looking history, while also noting that rebasing changes the apparent history. The choice depends on the workflow and collaboration rules.

VI. MOVING AROUND THE COMMIT TREE

A. Detached HEAD

```
git checkout <commit>
```

Checking out a commit directly attaches HEAD to that commit instead of to a branch. This is called detached HEAD.

```
Before: HEAD → main → C3

After:  HEAD → C2
                ↑
              main → C3
```

If you create commits while detached, no normal branch automatically moves with you. If the work matters, create or move a branch to preserve it.

B. Relative References: ^

```
git checkout main^
```

The ^ operator means the parent of a commit. main^ means the first parent of main. main^^ means the parent of the parent.

```
git checkout HEAD^
```

HEAD^ moves one parent backward from the current commit.

C. Relative References: ~

```
git checkout HEAD~4
```

The ~ operator moves back through generations. HEAD~4 means four parent steps backward along the first-parent path.

D. Multiple Parents

```
git checkout main^2
```

For a merge commit, ^2 selects the second parent. Without a number, ^ follows the first parent.

```
git checkout HEAD~^2~2
```

Relative-reference operators can be chained to navigate complicated commit graphs.

VII. MOVING BRANCH POINTERS

A. Force a Branch to a Commit

```
git branch -f main HEAD~3
```

This forcibly moves main to the commit three generations behind HEAD. The Learn Git Branching lesson uses this to teach that branches are movable references.

Note: real Git does not allow you to use git branch -f to move the branch that is currently checked out.

VIII. REVERSING CHANGES

A. Reset

```
git reset HEAD~1
```

Reset moves the current branch reference backward. In the Learn Git Branching model, this makes the branch appear as if the later commit had not been made. It is therefore a history-rewriting operation.

```
Before: main → C2 → C1

After reset HEAD~1:
main → C1

C2 may still exist as an object,
but main no longer points to it.
```

B. Revert

```
git revert HEAD
```

Revert creates a new commit whose changes undo the selected commit. Unlike reset, the existing history remains visible and a new reversing commit is added.

```
C1 → C2 → C2'
          ↑
        main

C2' reverses the changes introduced by C2.
```

The Learn Git Branching lessons emphasize reset for local history rewriting and revert when the reversal needs to be shared with others through a remote.

IX. REMOTE REPOSITORIES

A. What Is a Remote?

A remote repository is another copy of a Git repository, usually hosted elsewhere and reachable through a network. Remotes let collaborators exchange commits and also provide another copy of project history.

GitHub is a hosting service commonly used to host remote Git repositories. Git itself is the version-control system; GitHub is a service that can host and collaborate around Git repositories.

B. Clone

```
git clone <repository-url>
```

In ordinary Git, clone creates a local copy of a remote repository. It also sets up remote-tracking information, commonly using the remote name origin.

```
REMOTE REPOSITORY
       |
    clone
      ↓
LOCAL REPOSITORY
  main + origin/main
```

C. The Remote Name origin

The Learn Git Branching visualizer shortens origin to o/ because of UI space. In normal Git, origin is the conventional name automatically assigned to the remote created by clone.

```
git remote
git remote -v
git remote add origin <repository-url>
git remote remove origin
```

They are useful when creating a local repository and connecting it to an existing GitHub repository.

X. REMOTE BRANCHES AND origin/main

A. What origin/main Means

origin/main is a remote-tracking branch stored in your local repository. It represents your local knowledge of the main branch on the remote, as of the last time Git updated that information.

```
REMOTE                    LOCAL
main                       main
  ↓                         ↓
 C5                         C4
                             ↑
                        origin/main
```

B. Remote Branches Are Not Local Working Branches

Remote-tracking branches are references in your local repository. Checking one out directly places HEAD into detached mode; you normally create a local branch to do new work.

```
git checkout origin/main
git checkout -b feature origin/main
```

The second command creates a feature from the remote-tracking reference and establishes it as a branch that can be worked on.

XI. FETCH

A. Basic Fetch

```
git fetch
```

Fetch downloads commits that the remote has but the local repository does not, then updates remote-tracking branches such as origin/main. It does not automatically move your local main branch or rewrite your working files.

```
Before:
origin/main → C2
main        → C2

After remote gets C3, then git fetch:
origin/main → C3
main        → C2
```

fetch updates your knowledge of the remote; it does not integrate that work into your current local branch.

B. Fetch a Particular Remote Branch

```
git fetch origin foo
```

This fetches the remote foo branch and updates the corresponding remote-tracking reference, normally origin/foo.

C. Fetch with a Refspec

```
git fetch origin <source>:<destination>
```

For fetch, the source is a place on the remote and the destination is a local reference. The source:destination form gives explicit control over where the fetched commits are placed.

```
git fetch origin C2:bar
```

XII. PULL

A. What Pull Does

```
git pull
```

The core Learn Git Branching model is: git pull ≈ git fetch followed by git merge of the fetched work. It is a convenience command for obtaining remote changes and incorporating them.

```
git fetch; git merge origin/main
git pull
```

These represent the same basic workflow when the relevant tracking information points to origin/main.

B. Pull with Rebase

```
git pull --rebase
```

This is the shorthand workflow for fetching the remote changes and rebasing your local work on top of them.

```
git fetch; git rebase origin/main
```

C. Pull with Arguments

```
git pull origin main
git pull origin main:foo
```

XIII. PUSH

A. Basic Push

```
git push
```

Push uploads local commits to a remote and updates the corresponding remote branch. If the current branch has tracking information, Git can infer the destination.

```
git push origin main
```

This explicitly tells Git to push local main to main on the remote named origin.

```
LOCAL                          REMOTE
main → C3   →  main → C3
       ↑                         ↑
      C2                        C2
```

B. Push with a Source and Destination

```
git push origin <source>:<destination>
```

The colon refspec explicitly separates where commits come from locally and where they should go on the remote.

```
git push origin foo^:main
git push origin main:newBranch
```

The second example creates or updates a remote branch named newBranch from local main.

C. Deleting a Remote Branch with an Empty Source

```
git push origin :foo
```

,an empty source means nothing is being pushed to foo; this deletes the remote branch foo. Modern Git also commonly uses the clearer form git push origin --delete foo.

XIV. DIVERGED HISTORY

A. Why Push Can Be Rejected

Suppose the remote moved forward while you were working locally. Your local commit is then based on an older remote commit. Git may reject a normal push because blindly moving the remote backward could discard someone else's work.

```
Remote:  C1 → C2
              ↑
Local:   C1 → C3
```

B. Rebase Before Push

```
git fetch; git rebase origin/main; git push
```

The local work is replayed on top of the updated remote history and can then be pushed.

C. Merge Before Push

```
git fetch; git merge origin/main; git push
```

Instead of rewriting the local branch with a rebase, merge the remote work into it and then push the resulting history.

D. Shorter Workflows

```
git pull --rebase; git push
git pull; git push
```

The first uses fetch + rebase; the second uses fetch + merge

XV. REMOTE TRACKING

A. Tracking Relationship

A local branch can track a remote-tracking branch. This gives Git an implied destination for push and an implied integration target for pull.

```
git checkout -b foo origin/main
```

This creates foo from origin/main and establishes the tracking relationship.

```
git branch -u origin/main foo
git branch -u origin/main
```

The -u option sets the upstream/tracking branch. The first form explicitly names foo; the second can be used when foo is currently checked out.

B. Why Tracking Matters

```
HEAD → main → C5
               |
               | tracks
               ↓
         origin/main → C5
```

When tracking is configured, commands such as git push and git pull can often infer which remote branch is intended.

XVI. FEATURE-BRANCH WORKFLOW

A common collaborative pattern is to keep main as the integration branch and do feature work on separate branches. Once the feature is ready, it can be integrated into main and published to the remote.

```
main → C1 → C2
             \
              feature → C3 → C4
```

```
git checkout -b feature
git commit
git commit
git checkout main
git pull --rebase
git merge feature
git push
```

The exact workflow depends on the team's collaboration policy.

XVII. CHERRY-PICK

A. Copy Specific Commits

```
git cherry-pick <commit>
git cherry-pick C2 C4
```

Cherry-pick copies selected commits and places the copies below the current HEAD. It is useful when you want a specific piece of work without integrating an entire branch.

```
side: C1 → C2 → C3

main: C1 → C4

checkout main
git cherry-pick C2

main: C1 → C4 → C2'
```

The new commit has a new identity because it is a new commit object, even though it represents the changes from the selected commit.

XVIII. INTERACTIVE REBASE

A. Basic Form

```
git rebase -i HEAD~4
```

Interactive rebase opens a sequence of commits for controlled rewriting.

XIX. AMENDING A COMMIT

```
git commit --amend
```

Amend replaces the current commit with an updated version.

```
Before:
C1 → C2 → C3

amend C2:
C1 → C2' → C3'
```

Because the commit changes, later descendants may also need to be recreated when history is rewritten.

XX. TAGS

A. Creating a Tag

```
git tag v1 C1
git tag v1
```

Tags mark specific points in history. Unlike branches, tags are intended to remain fixed rather than move forward as new commits are created.

B. Checking Out a Tag

```
git checkout v1
```

Checking out a tag directly puts HEAD at that historical point rather than putting you on a normal working branch.

XXI. GIT DESCRIBE

```
git describe <ref>
```

```
git describe main
```

git describe reports a human-readable position relative to the nearest reachable tag. Learn Git Branching gives the conceptual output form:

```
<tag>-<numCommits>-g<hash>
```

Here, tag is the nearest tag in the relevant history, numCommits is the number of commits since that tag, and hash identifies the described commit.

XXII. ADVANCED REBASE PATTERNS

A. Rebasing Multiple Branches

The advanced Learn Git Branching level asks the learner to rebase several branches onto main and arrange the resulting commits in a required sequential order. The same principles apply: choose the correct branch, identify its commits, and rebase onto the intended base.

B. Multiple Parents

```
git checkout main^
git checkout main^2
git checkout HEAD~; git checkout HEAD^2; git checkout HEAD~2
git checkout HEAD~^2~2
```

The first-parent path is used by default for ^. Adding a number chooses a specific parent of a merge commit. The ~ operator continues along the first-parent path for the requested number of generations.

C. Branch Spaghetti

The final advanced lesson combines reordering, dropping, and transferring commits across multiple branches. The important tools are interactive rebase and cherry-pick, together with careful branch navigation.

XIII.

A. Starting from an Existing GitHub Repository

```
git clone <repository-url>
cd <repository-folder>
git status
git branch
git log --oneline
```

B. Creating a Feature

```
git switch -c feature
git status
git add .
git commit -m "Add feature"
git push -u origin feature
```

The -u option establishes the upstream relationship so later pushes and pulls can usually omit the remote and branch arguments.

C. Updating Your Feature Before Sharing

```
git fetch origin
git rebase origin/main
git push
```

If the feature branch was already pushed and rebased, a normal push may be rejected because the branch history was rewritten. The appropriate force-push strategy depends on the collaboration policy;

D. Integrating into main

```
git switch main
git pull --rebase
git merge feature
git push origin main
```

If the repository requires pull requests, the feature should instead be pushed and merged through that repository's review workflow.

XXIV.

A. main vs origin/main

main is your local branch. origin/main is your local remote-tracking reference for the remote's main branch.

B. git fetch vs git pull

fetch downloads and updates remote-tracking references. pull goes further by integrating the fetched work into the current branch.

C. git reset vs git revert

reset moves a branch reference and rewrites the visible local history. revert creates a new commit that undoes an earlier commit.

D. merge vs rebase

merge combines histories and may create a merge commit. rebase copies commits onto a new base and changes the apparent history.

E. branch vs commit

A commit is an object representing a snapshot/history point. A branch is a movable name pointing to a commit.

F. origin vs GitHub

origin is simply a remote name. GitHub is a hosting/collaboration service. A Git repository can have remotes hosted on many different services or servers.

XVI. FINAL COMMAND

```
git commit
git branch newImage
git checkout <name>
git checkout -b <branch>
git switch <name>
git switch -c <branch>
git merge <branch>
git rebase <branch>
git checkout <commit>
git checkout main^
git checkout HEAD^
git checkout HEAD~4
git branch -f main HEAD~3
git reset HEAD~1
git revert HEAD
git clone <repository-url>
git checkout origin/main
git checkout -b feature origin/main
git fetch
git fetch origin foo
git fetch origin <source>:<destination>
git pull
git pull --rebase
git pull origin main
git pull origin main:foo
git push
git push origin main
git push origin <source>:<destination>
git push origin foo^:main
git push origin main:newBranch
git push origin :foo
git branch -u origin/main foo
git branch -u origin/main
git cherry-pick <commit>
git cherry-pick C2 C4
git rebase -i HEAD~4
git commit --amend
git tag v1 C1
git tag v1
git describe <ref>
git describe main
git checkout main^2
git checkout HEAD~^2~2
```

XVII. REFERENCES

Source: https://learngitbranching.js.org/generatedDocs/levels.html